# ResumeLens: Formal Language-Based Résumé Screening

**Universidad Icesi**
Department of Computing and Intelligent Systems · Cali, Colombia

Juan Fernando Devia · Juan Jacobo Sandoval

**CyED III · 2026-2**
Computación y Estructuras Discretas III · Integrative Task 1

## Introduction

Recruiters review many résumés to check if candidates have the skills a job needs. This is hard to automate: the same skill is written in different ways (**JS**, **Javascript**, **JavaScript**), and each profile needs a different combination of skills.

> **Can formal language models decide if the skills in a résumé match a professional profile, no matter how the candidate writes them?**

We model each part of the problem with one formal model and join them in one Python pipeline for 4 profiles. ResumeLens does **not** rank candidates or make hiring decisions.

## Previous Work

- Regular expressions and finite automata define the same class: **regular languages** [1, 3].
- **Finite-State Transducers** (FSTs) are automata that also write output, used in translators and morphological analyzers [1, 3].
- **Context-Free Grammars** (CFGs) define nested structures and are the base of parsers and Domain-Specific Languages (DSLs) [2].

## Profiles

**Full Stack Developer**
JS/TS → Frontend → Backend → Database → Git

**Machine Learning Engineer**
Python → Data lib → ML lib → Storage → Git

**DevOps Engineer**
Scripting → Docker → (Kubernetes) → CI/CD → Cloud → Git

**Data Engineer**
Python/Scala → SQL → Processing → (Airflow) → Database → (Cloud) → Git

Fig. 1. Canonical order of each profile (dashed = optional).

## Methods

The input is a plain-text résumé. Each stage uses one formal model and gives its output to the next one (Fig. 2).

**Input:** `"3 years of experience developing web applications. Technical Skills: JS, React.js, NodeJS, Postgres, Git."`

**1 · Extract** — Regex · Python re
Finds candidate data and skills.
`JS, React.js, NodeJS, Postgres, Git`

→

**2 · Normalize** — FST · pyformlang
Maps each variant to one canonical name.
`JAVASCRIPT, REACT, NODE_JS, POSTGRESQL, GIT`

→

**3 · Recognize** — Automata · pyformlang
One automaton per profile accepts or rejects.
`FULL_STACK_DEVELOPER: ACCEPTED`

→

**4 · Validate** — CFG / DSL · textX
Validates the profile text and builds the view.
`valid → HTML`

Between 2 and 3: skills are filtered and sorted in each profile's canonical order

Fig. 2. ResumeLens pipeline with a real example from our tests.

### Design decisions

- The alphabet of the automata is the set of canonical names, so **one symbol = one skill**.
- Canonical sorting makes the result **independent of the order** used in the résumé.
- The 4 profiles share **one pipeline**; only the automaton changes.

### Formal models

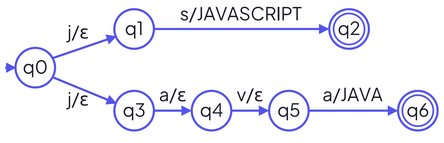



Fig. 3. FST M = (Q, Σ, Γ, δ, ω, $q_0$, F): paths for "js" and "java". Σ = characters, Γ = canonical names, ε = no output.

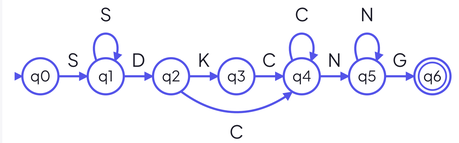



Fig. 4. DevOps DFA M = (Q, Σ, δ, $q_0$, F). S = Python/Bash, D = Docker, K = Kubernetes, C = CI/CD, N = cloud, G = Git.

| Profile | Type | Why |
|---|---|---|
| Full Stack | **DFA** | One next state per (state, symbol) |
| ML Eng. | **NFA** | δ($r_1$, PANDAS) = {$r_1$, $r_2$} |
| DevOps | **DFA** | Optional step uses its own symbol |
| Data Eng. | **ε-NFA** | ε skips Airflow and cloud |

Table 1. Automaton type of each profile.

```
Resume = "candidate", STRING, "{",
  Contact, Experience, {Experience},
  {Education}, Skills, Results, "}" ;
Skills = "skills", "{", Skill,
         {",", Skill}, "}" ;
Status = "ACCEPTED" | "REJECTED" ;
Skill  = /[A-Z][A-Z0-9_]*/ ;
```

Fig. 5. Part of the profile grammar (EBNF, textX).

## Results

**8/8** example résumés classified as expected

**48/48** automated tests passed (pytest)

| Profile | Accepted | Rejected (missing) |
|---|---|---|
| Full Stack | ✓ Wednesday | ✗ Enid (backend) |
| ML Eng. | ✓ Mary Jane | ✗ Harry (ML lib) |
| DevOps | ✓ Tony | ✗ Bruce (CI/CD) |
| Data Eng. | ✓ Natasha | ✗ Peter (database) |

Table 2. End-to-end results with the repository résumés.

Each rejected résumé misses exactly one required category. The grammar rejected invalid profile texts, such as `skills { git }` (lexical error) or `MAYBE` as a status (syntax error), and showed the line and column.

## Conclusions

- Yes, for our 4 profiles: the combination of regex, FST, automata and a CFG gives a decision that **does not depend on spelling or order**.
- Each model has one job: **find**, **translate**, **recognize** and **validate**.
- Limitations: unknown skill names are ignored, only English plain-text résumés, and the grammar checks structure, not meaning.

### Future research

- Load profiles and synonyms from files.
- Report the missing category of a rejected profile.

## References

1. J. E. Hopcroft, R. Motwani, J. D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Prentice Hall, 2006.
2. R. Lämmel. *Software Languages*. Springer, 2018.
3. R. de Castro. *Teoría de la Computación*. Universidad Nacional, 2004.

### Acknowledgments

We thank the CyED III teaching team. Generative AI was used as a collaboration tool (course level 3); prompts are in each author's AI log. All figures are our own.
Code: github.com/Duvias/Integradora-Discretas-3